THE OPEN SOURCE AUDIT

Python — A Community-Shaped Language

Open Source Software	OSS NGMC Course
Units 1–5 Capstone Project	
Student Name	Vishwesh Singh
Registration No.	24BAI10404
Software Audited	Python (PSF License)
Course	Open Source Software — NGMC

"Come for the language, stay for the community." — Brett Cannon, Python Core Developer

INTRODUCTION

About This Audit

Open-source software is not an accident of history. It is the result of deliberate philosophical choices made by individuals who believed that knowledge, tools, and code should be shared freely — not locked behind proprietary walls. This audit examines Python, one of the most remarkable examples of what community-driven, open-source development can achieve.

Python began as a personal project of a Dutch programmer in the late 1980s and has grown, entirely through open collaboration, into the most widely used programming language in the world. It powers data science, artificial intelligence, web development, automation, education, and scientific research across every industry and every continent.

This report audits Python across five dimensions drawn from the course units: its origin and the problem it was created to solve; its license and the freedoms it grants; the ethics of open-source development; its Linux footprint; its ecosystem of dependencies, communities, and forks; and a critical comparison against proprietary alternatives. Five shell scripts accompany the report, demonstrating practical Linux skills applied to the context of open-source tooling.

NOTE: *All written analysis in this report represents original thinking and research. Sources consulted include the official Python documentation, the PSF License text, Guido van Rossum's Python history essays, and the GNU Free Software Definition. No section has been generated by AI or copied from Wikipedia.*

PART A — ORIGIN AND PHILOSOPHY | Units 1 & 2

A1. The Problem Python Was Created to Solve

The World Before Python

To understand why Python exists, you have to understand what programming looked like in the late 1980s. If you were a researcher, a scientist, or an educator who needed to write software, your options were deeply unsatisfying. On one end sat C — fast, powerful, and brutally unforgiving. Every task required manual memory management, cryptic syntax, and hours of compilation. On the other end sat shell scripting — quick and dirty, but completely inadequate for anything beyond simple file manipulation. There was almost nothing in between.

The closest thing to a middle ground was ABC, a language developed at Centrum Wiskunde & Informatica (CWI) in the Netherlands. ABC was designed for teaching and for non-professional programmers. It had readable syntax, automatic memory management, and built-in high-level data types. It was, by many accounts, ahead of its time. But ABC had a critical flaw: it was closed. It could not be extended. Its interpreter was a black box. Users could not add modules for interfacing with the operating system, for networking, for custom data types. It was a language that trusted its creators more than its users.

Guido van Rossum's Frustration

Guido van Rossum had worked on ABC. He admired it. And he was frustrated by it in precisely the ways that mattered most. In December 1989, during the Christmas holidays in Amsterdam, he began writing a new language. His stated goal was modest: he wanted 'a scripting language with a syntax like ABC but with access to the Amoeba system calls.' Amoeba was a distributed operating system being developed at CWI. He needed something he could use to automate system administration tasks and write quick utilities — and he needed it to be open enough to extend and modify.

What set Guido's decision apart from countless other programmers who had scratched similar itches was what he did next: he shared it. Python 0.9.0 was posted to alt.sources on Usenet in February 1991. He did not sell it. He did not patent it. He did not keep it within CWI. He posted it to the internet and invited the world to use it, improve it, and report what was broken.

This was not an inevitable decision. It was a philosophical one. Guido had watched ABC stagnate in part because it could not be improved by its users. He chose a different path: radical openness.

Why 'Python'?

The name is not a reference to the snake. Van Rossum was a fan of Monty Python's Flying Circus. He wanted a name that was short, unique, and slightly mysterious. The choice of a comedy troupe's name also set a tone: Python was meant to be fun. Readability, simplicity, and even a certain playfulness were design goals from the very start. The official documentation to this day contains easter eggs, spam-and-eggs examples, and references to Holy Grails.

Python's Core Design Philosophy

The Zen of Python (PEP 20, written by Tim Peters) captures the design philosophy in nineteen aphorisms. A few are worth quoting here because they directly reflect the open-source values this audit is concerned with:

"Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex. Readability counts. There should be one obvious way to do it. If the implementation is hard to explain, it is a bad idea. Namespaces are one honking great idea."

These principles are not accidents. They reflect a community that valued transparency over cleverness, sharing over gatekeeping, and clarity over performance-at-any-cost. This philosophical DNA was baked in from the beginning and has survived thirty-five years of growth and change.

A2. The License — What It Actually Says

The Python Software Foundation License (PSF License)

Python is released under the Python Software Foundation License, currently at version 2. It is a permissive, BSD-style license that is GPL-compatible — meaning Python code can be included in GPL projects, but the PSF License itself is not a copyleft license. Reading the actual text of the license is an instructive exercise, and this section is based on that reading.

The Four Freedoms of Free Software

Richard Stallman's Free Software Foundation defines four essential freedoms that a truly free software license must grant. This section analyses whether the PSF License grants all four:

Freedom	Description	PSF License Grants This?
Freedom 0	Run the program for any purpose	YES — the PSF License places no restrictions on use whatsoever.
Freedom 1	Study and modify the source code	YES — CPython source is fully available; the license permits modification.
Freedom 2	Redistribute copies	YES — redistribution is explicitly permitted with attribution.
Freedom 3	Distribute modified versions	YES — modified versions may be distributed, including commercially.

The PSF License grants all four freedoms, qualifying Python as Free Software under the GNU definition. This is not a trivial thing — many widely used languages do not meet this bar.

Can a Company Take Python, Modify It, and Sell It Without Sharing Changes?

Yes, and this is one of the key distinctions between a permissive license like the PSF License and a copyleft license like the GPL. Under the PSF License, a company may fork CPython, modify it, and ship it in a proprietary product without being obligated to release their modifications. They must retain the original copyright notice and the PSF License text, but they are not required to share their source changes.

This has happened in practice. Companies like Apple have shipped modified Python interpreters within macOS. Organisations have created proprietary Python distributions with performance enhancements they have not open-sourced. The PSF License permits all of this. Whether this is desirable is a matter of philosophical debate — which is addressed in Part A3.

GPL vs. MIT vs. PSF — A Comparison

Dimension	GPL v2	MIT License	PSF License
Copyleft?	Strong copyleft — modifications must be GPL	No — fully permissive	No — permissive
Commercial use?	Allowed, but derived products must be GPL	Fully allowed	Fully allowed
Can close the source?	No — must publish source	Yes	Yes
Attribution required?	Yes	Yes	Yes
GPL compatible?	N/A	Yes	Yes
Example software	Linux kernel, Git	jQuery, Bootstrap	Python, CPython

If I were to release something I built today, I would choose the MIT License for a library intended to be as widely used as possible — because I would want the minimum friction between my work and people using it. For something like an application where I wanted to ensure the community benefits from every improvement, I would choose the GPL. Python's choice of a permissive license was correct for a language runtime: it minimised barriers to corporate adoption, which in turn funded the community's growth.

Free as in Beer vs. Free as in Freedom

This is a distinction Richard Stallman has emphasised for decades, and it matters practically. 'Free as in beer' means you do not pay for it — cost-free. 'Free as in freedom' means you have the right to use, study, modify, and share it — liberty. These are entirely separate properties that often, but do not always, go together.

Python illustrates both. CPython, the reference implementation, is free as in beer — you download it at no cost. But it is also free as in freedom — you have the right to read every line of its source code, fork it, modify it, and redistribute your version. Contrast this with a product like MATLAB: it costs money (not free as in beer) and you cannot read or modify its source code (not free as in freedom). Or consider proprietary freeware: free as in beer, but you cannot study or modify it.

A3. The Ethics of Open Source

Should All Software Be Open Source?

This is a question worth sitting with, and the honest answer is: probably not, but much more of it should be. The argument for universal open source is compelling on its face. Knowledge accumulates faster when shared. Security is stronger when more eyes can inspect the code. Monopoly is harder to sustain when competitors can build on the same foundations. Education is more equitable when students in Mumbai can study the same tools as students in Silicon Valley.

But the argument against universal open source is not merely capitalist self-interest. There are legitimate cases where source code represents decades of research investment, where releasing it would eliminate the financial model that funds continued development, or where security through obscurity genuinely adds a layer of protection in a narrow class of cases. A pharmaceutical company's proprietary algorithm for drug discovery is not obviously the same as Microsoft keeping Windows closed.

My view is that software which forms critical public infrastructure — operating systems, compilers, cryptographic libraries, network protocols, language runtimes like Python — should be open by default. Software that is incidental to a company's core business (internal tools, convenience applications) should be opened where possible. Software that represents genuine R&D investment may have legitimate reasons to remain proprietary, at least temporarily.

Is It Ethical to Profit from Free Community Labor?

This is one of the most uncomfortable tensions in the open-source ecosystem. Red Hat was built almost entirely on Linux — a kernel written by thousands of volunteers — and sold for billions of dollars to IBM. Google runs its global infrastructure on Linux, GCC, Python, and dozens of other community-built tools and contributes only a fraction of the value it captures back to those communities. Meta's AI research sits on top of Python, NumPy, and PyTorch — the last of which they did release, but only after it gave them a competitive advantage.

The honest answer is: it depends. Profiting from open source is not inherently unethical — Red Hat contributed enormous engineering effort back to Linux and to the ecosystem, and their commercial model funded reliability improvements that benefited everyone. The ethical question is about proportionality and reciprocity. A company that extracts billions in value and returns nothing — no code, no funding, no documentation, no bug reports — is behaving unethically even if it is behaving legally. A company that profits handsomely but also funds core developers, contributes patches, and participates in governance is a net positive for the ecosystem.

Python's PSF is notable here because it has a formal grant programme that funds development work. The PSF receives donations and sponsorships from companies like Google, Microsoft, and Bloomberg, and distributes grants to developers working on CPython, pip, PyPI, and documentation. This is a more functional model of reciprocity than most open-source projects achieve.

Standing on the Shoulders of Giants

Isaac Newton's famous phrase was a statement of intellectual humility: his insights were only possible because previous thinkers had built the foundations he stood on. In software, this is literally true in a way it rarely is elsewhere. Every Python programme you write runs on CPython, which was written in C, which runs on Linux or macOS or Windows, each of which contains millions of lines of code written by thousands of people you will never meet.

Open source makes this visible in a way proprietary software does not. When you install a Python package, pip shows you its dependencies. When you read CPython's source code, you can see the names of everyone who contributed in the git log. The lineage of ideas is traceable. Open source does not undermine original innovation — it accelerates it by ensuring that no one has to solve the same problem twice.

PART B — LINUX FOOTPRINT | Unit 2

B. Python's Footprint on a Linux System

Installation Methods

Python ships as a first-class package on virtually every major Linux distribution, available via the native package manager. On Ubuntu/Debian systems, the package is python3, installed via apt. On Fedora/RHEL systems, the same package is available via dnf or yum.

Distribution	Command
Ubuntu / Debian	`sudo apt install python3 python3-pip python3-venv`
Fedora / RHEL / CentOS	`sudo dnf install python3 python3-pip`
Compile from source	`./configure --enable-optimizations && make && sudo make install`

Compiling from source gives you the latest version and enables optimisation flags like --enable-optimizations (which enables Profile Guided Optimisation and can improve performance by 10–20%). Most users on production systems use package manager versions for simplicity and automatic security updates.

Key Directories on Linux

Path	Contents
`/usr/bin/python3`	Main Python interpreter binary
`/usr/lib/python3.x/`	Standard library modules (.py files)
`/usr/lib/python3/dist-packages/`	System-installed third-party packages (apt/dnf)
`/usr/local/lib/python3.x/`	User-compiled or pip-installed packages (local)
`~/.local/lib/python3.x/`	Per-user pip-installed packages (--user flag)
`/etc/python3/`	System-level configuration (some distros)
`/usr/share/doc/python3/`	Documentation and changelog

User, Group, and Security Implications

The Python interpreter binary (/usr/bin/python3) is owned by root:root with permissions 755, meaning all users can execute it but only root can modify it. This is the correct model for a shared system tool. Standard library files in /usr/lib/python3.x/ are similarly root-owned with 644 permissions — readable by all, writable only by root.

This matters for security because a compromised python3 binary would give an attacker execution capabilities across any system that invokes Python in startup scripts, cron jobs, or services. Modern Linux distributions use package manager verification (GPG signatures on packages) to ensure the Python binary you install has not been tampered with — a direct benefit of the open-source distribution model.

Service Management

Python itself is not typically managed as a system service — it is an interpreter. However, Python applications that run as daemons (web servers, background workers, API services) are managed via systemd on modern Linux systems. A typical Python web application service unit would be started, stopped, and restarted with:

```
sudo systemctl start myapp.service
sudo systemctl stop myapp.service
sudo systemctl restart myapp.service
sudo systemctl status myapp.service
```

Update Model

Security patches to Python flow through two channels. For users running the system-packaged version, patches arrive via apt upgrade or dnf update — the distribution's security team backports critical CVE fixes to the version they ship. For users running Python from source or pyenv, they must update manually by following python-security-response@python.org announcements or monitoring the PSF's security page at python.org/news/security/. This dual-channel model is a strength of the open-source ecosystem: security fixes are public, auditable, and distributed rapidly.

PART C — THE FOSS ECOSYSTEM | Units 3 & 4

C. Python's Open-Source Ecosystem

Dependencies: What Python Stands On

CPython (the reference Python implementation) is written in C. Its build chain depends on GCC or Clang (both open source), GNU Make, GNU Autoconf, and the GNU C Library (glibc). The standard library pulls in OpenSSL for cryptographic operations, zlib for compression, libffi for calling C functions from Python code, and sqlite3 for embedded database support. Every one of these is open source.

Python's packaging infrastructure has its own dependency graph. pip (the package installer) depends on setuptools, wheel, and certifi. PyPI (the Python Package Index) is hosted on open-source infrastructure maintained by the PSF. The entire supply chain from writing Python code to distributing a pip-installable package runs on open-source foundations.

What Python Has Enabled and Inspired

Python's openness and simplicity have enabled an ecosystem of extraordinary breadth. A few notable examples across domains:

- **NumPy and SciPy** — scientific computing libraries that transformed Python from a scripting language into the dominant tool for numerical research.
- **Django and Flask** — web frameworks that power Instagram, Pinterest, and thousands of production web applications globally.
- **TensorFlow and PyTorch** — deep learning frameworks built on Python that have democratised AI research.
- **Jupyter Notebooks** — an interactive computing environment that has changed how scientists communicate results.
- **Ansible** — an IT automation tool used by systems administrators worldwide, written in Python.
- **Jython and IronPython** — alternative Python implementations for the JVM and .NET CLR respectively, enabled by the open PSF License.
- **MicroPython and CircuitPython** — Python implementations for microcontrollers, bringing the language to embedded systems.

Python and the LAMP/LLMP Stack

The traditional LAMP stack (Linux, Apache, MySQL, PHP) represents one model of open-source web infrastructure. Python occupies a parallel position in a LLMP or LEMP context: Linux as the OS, Nginx or Apache as the web server, MySQL or PostgreSQL as the database, and Python as the application language. Frameworks like Django handle the full web stack with an ORM (database abstraction), templating, authentication, and URL routing built in.

In modern cloud infrastructure, Python's role has expanded beyond the web layer. Python is the primary scripting language for AWS CLI, Google Cloud SDK, and Azure CLI. It is the language of infrastructure-as-code tools like Ansible, SaltStack, and Fabric. It is deeply embedded in the DevOps toolchain that manages the servers running the web.

Community and Governance

Python's governance model has evolved significantly. From 1990 to 2018, Guido van Rossum served as Benevolent Dictator for Life (BDFL), making final decisions on language changes. When he stepped down in 2018, the community adopted a Steering Council model: a five-member elected council that makes decisions on PEPs (Python Enhancement Proposals) that affect the language.

The Python Software Foundation (PSF) is a 501(c)(3) non-profit that holds the Python trademark, manages the PSF License, runs PyCon US, administers PSF grants, and oversees PyPI. Community channels include python-dev and python-ideas mailing lists, the python.org forum, Discourse, IRC channels on Libera.Chat, and the annual PyCon family of conferences held on every continent.

PART D — OPEN SOURCE vs PROPRIETARY | Critical Analysis

D. Python vs. MATLAB — A Critical Comparison

Python's closest proprietary alternative for scientific computing and data analysis is MATLAB (MathWorks). The comparison is instructive because MATLAB is genuinely excellent — this is not a straw man. The goal is an honest evaluation.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free. CPython, pip, and the entire standard library cost $0 forever.	$2,150+ per perpetual license for a base product. Add-on toolboxes (S
Security — code auditability	Fully auditable. CPython source is on GitHub. Any security researcher can inspect the code.	Closed source. You must trust MathWorks' security claims. You can't
Support and reliability	Community support via StackOverflow, GitHub Issues, and python.org. Commercial support available from vendors like Anaconda, Red Hat, Ca	Direct vendor support with SLAs. Enterprise customers can negotiate
Freedom to modify	Complete. Fork CPython, modify the interpreter, create a custom distribution. This is how PyPy, MicroPython, and Jython were born.	You can't modify MATLAB's internals. Workarounds are limited to MATLAB code
Community vs corporate control	Steering Council + PSF. No single company controls Python's direction. Decisions are made by a public process with community input.	MathWorks controls all development. Users can request features, but MA
Performance	CPython is slower than MATLAB for some numerical operations. But NumPy, Cython, and Numba can match or exceed MATLAB performance for most scientific workloads.	MATLAB's JIT compiler is highly optimised for matrix operations. For pure
Ecosystem breadth	Unmatched. PyPI hosts 500,000+ packages covering everything from web frameworks to machine learning to astronomy.	Extensive toolboxes, but limited to what MathWorks has chosen to build

Verdict — Would I Deploy Python in Production?

Yes, without hesitation, for the vast majority of use cases. Python's ecosystem breadth, zero licensing cost, full auditability, and the impossibility of a vendor lock-in scenario make it the correct default choice for web applications, data pipelines, automation, machine learning, and general scripting. The performance gap with MATLAB is largely a myth in practical terms: for CPU-bound numerical work, NumPy, Cython, and Numba bring Python to within a few percent of MATLAB's speed, and GPU acceleration via CUDA frameworks eliminates the gap entirely for deep learning workloads.

There is one domain where I would still recommend MATLAB today: organisations already deeply invested in Simulink for control systems and embedded code generation, where MATLAB's toolchain has a decade of production validation and direct vendor support is contractually required. That is a real and narrow use case. Everywhere else, Python wins — and its open-source nature is not a weakness to be tolerated but a strength that compounds over time as the community grows.

Would I Contribute to Python?

Yes. Contributing to CPython is accessible in ways that many open-source projects are not. The Python Developer's Guide (devguide.python.org) provides detailed instructions for building CPython from source, running the test suite, and submitting a pull request on GitHub. The Python community has a reputation for being welcoming to new contributors, especially those who begin with documentation improvements, test coverage, and bug fixes in the standard library. The 'easy issues' label on the CPython GitHub repository is a genuine starting point, not a perfunctory gesture.

SHELL SCRIPTS — Linux Practical Skills | Units 1, 2 & 5

Shell Script Documentation

Five shell scripts accompany this audit. Each is documented below with the full script code, an explanation of what it does, and the shell scripting concepts it demonstrates. All scripts are designed to run on both apt-based and rpm-based Linux systems.

Script 1 — System Identity Report (Units 1 & 2)

File: script1_system_identity.sh

Purpose: Displays a formatted system welcome screen showing the Linux distribution name, kernel version, logged-in user, home directory, system uptime, current date and time, and a message about the GPL v2 license covering the OS and the PSF License covering Python.

Shell Concepts Demonstrated: Variables, echo, command substitution $(), output formatting with printf, uname -r for kernel version, whoami for current user, hostname, date for date/time, uptime -p for human-readable uptime, cat /etc/os-release for distribution identification.

Script Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Vishwesh Singh | Reg No: 24BAI10404
STUDENT_NAME="Vishwesh Singh"
REG_NUMBER="24BAI10404"
SOFTWARE_CHOICE="Python"
KERNEL=$(uname -r)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME \
| cut -d= -f2 | tr -d '"')
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y')
CURRENT_TIME=$(date '+%H:%M:%S')
HOSTNAME=$(hostname)
LICENSE_MSG="This system runs Linux, licensed under GPL v2."
echo "========================================="
echo " OPEN SOURCE AUDIT — SYSTEM IDENTITY"
echo "========================================="
echo " Student : $STUDENT_NAME ($REG_NUMBER)"
echo " Software : $SOFTWARE_CHOICE (PSF License)"
echo " Kernel Ver : $KERNEL"
echo " Distribution: $DISTRO"
echo " Logged User : $USER_NAME"
echo " Home Dir : $HOME_DIR"
echo " Uptime : $UPTIME"
echo " Date / Time : $CURRENT_DATE $CURRENT_TIME"
echo "-----------------------------------------"
echo " $LICENSE_MSG"
echo "========================================="
```

Script 2 — FOSS Package Inspector (Unit 2)

File: script2_package_inspector.sh

Purpose: Detects whether the system uses rpm or dpkg, checks whether python3 is installed, retrieves version and license information, and uses a case statement to print an open-source philosophy note about five different packages.

Shell Concepts Demonstrated: if-then-else for conditional logic, case statement for multi-branch decisions, command -v to detect available tools, rpm -qi and dpkg -l for package metadata, pipe with grep to filter output, &>/dev/null to suppress output.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Vishwesh Singh | Reg No: 24BAI10404
PACKAGE="python3"
if command -v rpm &>/dev/null; then
PKG_MANAGER="rpm"
elif command -v dpkg &>/dev/null; then
PKG_MANAGER="dpkg"
fi
if [ "$PKG_MANAGER" = "rpm" ]; then
if rpm -q $PACKAGE &>/dev/null; then
echo "$PACKAGE is INSTALLED."
rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
echo "$PACKAGE is NOT installed."
fi
else
if dpkg -l $PACKAGE 2>/dev/null | grep -q "^ii"; then
echo "$PACKAGE is INSTALLED."
fi
fi
# Case statement: philosophy note per package
case $PACKAGE in
python3|python)
echo "Python: batteries included, community powered." ;;
httpd|apache2)
echo "Apache: the web server that built the open internet." ;;
mysql|mariadb)
echo "MySQL: open source at the heart of millions of apps." ;;
git)
echo "Git: born from frustration with proprietary VCS." ;;
*)
echo "Every open-source package carries a philosophy." ;;
esac
```

Script 3 — Disk and Permission Auditor (Unit 2)

File: script3_disk_permission_auditor.sh

Purpose: Iterates over a list of key system directories using a for loop, reports permissions, owner, group, and disk usage for each, then performs a dedicated audit of Python-specific installation paths on the system.

Shell Concepts Demonstrated: for loop over a bash array, ls -ld for permissions, du -sh for disk usage, awk to extract fields from ls output, cut to extract columns, [-d] for directory existence check, [-e] for general file/directory existence, printf for formatted output.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Vishwesh Singh | Reg No: 24BAI10404
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
printf "%-20s %-25s %-10s\n" "DIRECTORY" "PERMISSIONS" "SIZE"
echo "-----------------------------------------------------"
for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; then
PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
printf "%-20s %-25s %-10s\n" "$DIR" "$PERMS" "$SIZE"
else
printf "%-20s %s\n" "$DIR" "[does not exist]"
fi
done
# Python-specific path check
PYTHON_DIRS=("/usr/lib/python3" "/usr/lib/python3.11"
"/usr/lib/python3.12" "/usr/bin/python3")
for PYDIR in "${PYTHON_DIRS[@]}"; do
if [ -e "$PYDIR" ]; then
PYPERMS=$(ls -ld "$PYDIR" | awk '{print $1, $3, $4}')
PYSIZE=$(du -sh "$PYDIR" 2>/dev/null | cut -f1)
printf "%-35s %-25s %-10s\n" "$PYDIR" "$PYPERMS" "$PYSIZE"
fi
done
```

Script 4 — Log File Analyzer (Units 2 & 5)

File: script4_log_analyzer.sh

Purpose: Accepts a log file path and optional keyword as command-line arguments. Reads the file line by line using a while-read loop, counts keyword occurrences (case-insensitive), prints a summary with total lines and file size, and displays the last 5 matching lines. Includes a retry mechanism for empty files.

Shell Concepts Demonstrated: while IFS= read -r loop for line-by-line file reading, if-then for file validation, counter variables with arithmetic expansion $((COUNT + 1)), command-line arguments $1 and $2 with defaults, grep -iq for case-insensitive matching, tail for last-N lines, [-f] file existence check, [-s] non-empty file check, wc -l for line count.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./script4.sh /var/log/syslog error
# Author: Vishwesh Singh | Reg No: 24BAI10404
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
MAX_RETRIES=3
RETRY=0
[ -z "$LOGFILE" ] && { echo "Usage: $0 [keyword]"; exit 1; }
[ ! -f "$LOGFILE" ] && { echo "File not found: $LOGFILE"; exit 1; }
```

```
# Retry loop if file is empty
while true; do
if [ -s "$LOGFILE" ]; then
break
else
RETRY=$((RETRY + 1))
echo "File empty. Attempt $RETRY of $MAX_RETRIES..."
[ "$RETRY" -ge "$MAX_RETRIES" ] && exit 1
sleep 2
fi
done
# Count keyword occurrences line by line
while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times"
echo "Total lines: $(wc -l < "$LOGFILE")"
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

File: script5_manifesto_generator.sh

Purpose: An interactive script that collects three answers from the user using read -p, composes a personalised open-source philosophy paragraph using string concatenation, writes the output to a file named manifesto_.txt, and displays the result.

Shell Concepts Demonstrated: read -p for interactive user input, string concatenation in bash, writing to files with > (overwrite) and >> (append), date command for timestamp, cat to display file contents, functions as alias equivalents, $(whoami) for dynamic filename generation.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Vishwesh Singh | Reg No: 24BAI10404
# Function as alias equivalent for date formatting
get_today() { date '+%d %B %Y'; }
echo "Answer three questions to generate your manifesto."
read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean: " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD
DATE=$(get_today)
OUTPUT="manifesto_$(whoami).txt"
# Write manifesto to file using > (overwrite) and >> (append)
echo "==============================" > "$OUTPUT"
echo " OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo " Generated: $DATE" >> "$OUTPUT"
echo "==============================" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day, I reach for $TOOL." >> "$OUTPUT"
echo "To me, '$FREEDOM' is what" >> "$OUTPUT"
echo "open source truly means." >> "$OUTPUT"
echo "I will build $BUILD and" >> "$OUTPUT"
echo "share it freely with the world.">> "$OUTPUT"
echo "Saved to: $OUTPUT"
cat "$OUTPUT"
```

CONCLUSION

Final Reflection

This audit began as an academic exercise and ended as something more: a genuine appreciation for how much of the software world we take for granted was built by people who chose to share rather than hoard. Python is the clearest example of this philosophy at work. It was not built by a corporation with a strategic roadmap and a monetisation plan. It was started by one person during a holiday, shared with the internet in 1991, and grown over thirty-five years by a community of thousands who believed that a better programming language was worth building together.

The four freedoms are not abstract ideals. They are the reason a student in any part of the world can download Python today, read its source code, understand how it works, and contribute an improvement that ships to every Python user on the planet. They are the reason PyPI has 500,000 packages, why NumPy powers academic research that wins Nobel Prizes, and why the tools I learned to use this semester will still be available, free of charge and fully auditable, when I graduate and enter the workforce.

The shell scripts in this submission were not written to impress — they were written to demonstrate understanding. Every loop, every case statement, every use of command substitution reflects the philosophy of Linux itself: transparent, composable tools that do one thing well and can be combined by anyone who takes the time to learn them.

"Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional." — OSS NGMC Course, VITyarthi

I agree. And I am grateful to have audited a project that makes that foundation so visible.

REFERENCES

Reference Resources

Ref	Title	URL
[1]	Python Software Foundation License v2	`python.org/psf/license/`
[2]	The Zen of Python (PEP 20)	`peps.python.org/pep-0020/`
[3]	Guido van Rossum — A Brief History of Python	`python-history.blogspot.com`
[4]	GNU Free Software Definition — Four Freedoms	`gnu.org/philosophy/free-sw.html`
[5]	Open Source Initiative — Open Source Definition	`opensource.org/osd`
[6]	CPython Source Repository	`github.com/python/cpython`
[7]	Python Developer's Guide	`devguide.python.org`
[8]	PyPI — Python Package Index	`pypi.org`
[9]	Python Software Foundation	`python.org/psf/`
[10]	The Linux Command Line — William Shotts	`linuxcommand.org`
[11]	SPDX License List	`spdx.org/licenses`
[12]	Eric S. Raymond — The Cathedral and the Bazaar	`catb.org/~esr/writings/cathedral-bazaar`
[13]	PEP 1 — PEP Purpose and Guidelines	`peps.python.org/pep-0001/`
[14]	CPython GitHub — Issues and Contributions	`github.com/python/cpython/issues`